Database Systems Overview

The Red Thread: From Theory to Practice

Comprehensive Course Summary

Contents

1 The Big Picture: Course Overview

The Database Journey

What this course teaches:

1. **Theory (Relational Algebra)**: Mathematical foundation for data manipulation
2. **Practice (SQL Basic & Advanced)**: Industry-standard language implementing the theory
3. **Design (ER Models)**: How to structure data effectively BEFORE writing SQL
4. **Normalization**: Remove redundancy, ensure data integrity
5. **Optimization**: Making queries fast and efficient
6. **Concurrency & Recovery**: Handling multiple users and failures

The Red Thread:
Requirements → ER Design → Relational Schema → SQL Implementation → Query Optimization → Physical Design → Transactions & Recovery

Key Insight: Design happens FIRST (ER Model), then implementation (SQL CREATE TABLE), then usage (SQL queries)!

How Topics Connect

Foundation Layer:

- **Relational Model**: Tables (relations), rows (tuples), columns (attributes)
- **Keys**: Primary keys uniquely identify rows, foreign keys link tables

Query Layer:

- **Relational Algebra**: Theoretical operations (, , , , , -)
- **SQL**: Practical implementation (SELECT, WHERE, JOIN, UNION, etc.)

Design Layer (BEFORE Implementation):

- **Requirements Analysis**: Identify nouns (entities) and verbs (relationships)
- **ER Modeling**: Design database structure (entities, relationships, cardinality)
- **Mapping ER to Relational**: Convert ER diagram to table schemas
- **Normalization**: Remove redundancy, prevent anomalies

Performance Layer:

- **Indexing**: Speed up searches (B+ trees, hash indexes)
- **Query Processing**: How databases execute queries efficiently

Reliability Layer:

- **Transactions**: ACID properties ensure data consistency
- **Concurrency Control**: Multiple users accessing data safely
- **Recovery**: Restoring database after crashes

2 Design to Implementation: ER Model → SQL

The 4-Step Database Design Process

Step 1: Requirements Collection & Analysis

- Talk to stakeholders, understand what data is needed
- **Heuristic:** Nouns → entities, Verbs → relationships
- Example: "Instructors *teach* courses" → `instructor` entity, `course` entity, `teaches` relationship

Step 2: Conceptual Design (ER Diagram)

- Create ER diagram with entities (rectangles), relationships (diamonds), attributes (ovals)
- Define cardinality (1:1, 1:N, M:N) and participation (total=double line, partial=single line)
- Independent of specific database system

Step 3: Data Model Mapping (ER → Relational Schema)

- Convert entities → tables, attributes → columns
- Primary keys (underlined in ER) → PRIMARY KEY in SQL
- Relationships → foreign keys or junction tables (for M:N)

Step 4: Physical Design (SQL Implementation)

- Write CREATE TABLE statements with constraints
- Add indexes for performance, configure storage

Key Point: Design (ER) comes BEFORE writing SQL code!

ER to SQL Mapping Examples

Entity → Table:
ER: instructor entity with attributes {ID, name, dept_name, salary}
SQL:

```
CREATE TABLE instructor (
    ID CHAR(5) PRIMARY KEY,
    name VARCHAR(20) NOT NULL,
```

```
    dept_name VARCHAR(20),
    salary NUMERIC(8,2)
);
```

1:N Relationship → Foreign Key:
ER: department (1) ← has → (N) employee
SQL: Add foreign key in employee table

```
CREATE TABLE employee (
    emp_id CHAR(5) PRIMARY KEY,
    name VARCHAR(20),
    dept_name VARCHAR(20),
    FOREIGN KEY (dept_name) REFERENCES department(dept_name)
);
```

M:N Relationship → Junction Table:
ER: student (M) ← takes → (N) course with attributes {grade, semester}
SQL: Create separate table for the relationship

```
CREATE TABLE takes (
    student_id CHAR(5),
    course_id VARCHAR(8),
    semester VARCHAR(6),
    grade VARCHAR(2),
    PRIMARY KEY (student_id, course_id, semester),
    FOREIGN KEY (student_id) REFERENCES student(ID),
    FOREIGN KEY (course_id) REFERENCES course(course_id)
);
```

Cardinality Determines Implementation

ER Cardinality	SQL Implementation
1:1	Foreign key in either table (or both)
1:N	Foreign key in the "many" side (N side)
M:N	Separate junction table with FKs to both entities

Total participation (double line): Add NOT NULL constraint or CHECK constraint to ensure every entity participates.

3 Relational Algebra SQL: The Connection

Theory Meets Practice

Core Principle: SQL implements Relational Algebra!

Relational Algebra		SQL
SELECT ()	→	`WHERE` clause
PROJECT ()	→	`SELECT` clause
CARTESIAN PRODUCT (×)	→	`FROM t1, t2`
THETA JOIN ()	→	`FROM t1, t2 WHERE t1.key=t2.key`
NATURAL JOIN ()	→	`NATURAL JOIN` or explicit join
UNION ()	→	`UNION`
INTERSECTION ()	→	`INTERSECT`
DIFFERENCE (-)	→	`EXCEPT`
RENAME ()	→	`AS` keyword
AGGREGATION ()	→	`GROUP BY` + aggregates

Key Insight: Understanding RA makes SQL intuitive!

3.1 Operation Mapping Examples

(SELECT) → WHERE

Relational Algebra:

$$\sigma_{\text{age}>21}(\text{Student})$$

SQL Equivalent:

```
SELECT *
FROM Student
WHERE age > 21;
```

What it does: Filters rows based on condition

(PROJECT) → SELECT

Relational Algebra:

$$\Pi_{\text{name, age}}(\text{Student})$$

SQL Equivalent:

```
SELECT name, age
FROM Student;
```

What it does: Selects specific columns, removes duplicates

Combined: + → WHERE + SELECT

Relational Algebra:

$$\Pi_{\text{name}}(\sigma_{\text{age}>21\wedge\text{major}=\text{'CS'}}(\text{Student}))$$

SQL Equivalent:

```
SELECT name
FROM Student
WHERE age > 21 AND major = 'CS';
```

Pattern: Inside-out in RA = bottom-up in SQL (FROM → WHERE → SELECT)

(JOIN) → Multi-table FROM + WHERE

Relational Algebra:

$$\text{Student} \bowtie_{\text{Student.ID}=\text{Enrollment.SID}} \text{Enrollment}$$

SQL Equivalent:

```
SELECT *
FROM Student, Enrollment
WHERE Student.ID = Enrollment.SID;
```

What it does: Combines tables based on matching keys

4 Key Concepts Dictionary

4.1 Fundamental Terminology

Database Basics

Relation	A table; a set of tuples with the same structure
Tuple	A row in a table; represents one entity/record
Attribute	A column in a table; a property of entities
Domain	The set of allowed values for an attribute (e.g., integers, strings)
Schema	Structure definition: relation name + attribute names + types
Instance	Actual data in the database at a specific moment
Superkey	Set of attributes that uniquely identifies tuples
Candidate Key	Minimal superkey (no subset is also a superkey)
Primary Key	The chosen candidate key (underlined in schema)
Foreign Key	Attribute referencing primary key of another relation
NULL	Special value representing unknown or missing data

4.2 Relational Algebra Operations

Unary Operations (1 input relation)

SELECT ()	Filter rows by condition; horizontal filtering
PROJECT ()	Select columns, remove duplicates; vertical filtering
RENAME ()	Change relation/attribute names

Binary Operations (2 input relations)

UNION ()	All rows from R or S (requires union-compatibility)
INTERSECTION ()	Rows in both R and S (requires union-compatibility)
DIFFERENCE (-)	Rows in R but not in S (requires union-compatibility)

CARTESIAN PRODUCT (×)
All possible (r,s) pairs from R and S

THETA JOIN ()
Cartesian product filtered by condition

NATURAL JOIN ()
Join on common attributes with equal values

LEFT OUTER JOIN ()
Keep all left rows, fill missing with NULL

RIGHT OUTER JOIN ()
Keep all right rows, fill missing with NULL

FULL OUTER JOIN
Keep all rows from both, fill missing with NULL

Grouping & Aggregation

AGGREGATION ()
Group tuples and apply aggregate functions

count(*)	Count tuples in group
sum(A)	Sum of attribute A values in group
avg(A)	Average of attribute A in group
min(A)	Minimum value of A in group
max(A)	Maximum value of A in group

4.3 SQL Key Concepts

SQL Query Components

SELECT	Specifies output columns (PROJECT in RA)
DISTINCT	Removes duplicate rows from result
FROM	Lists tables to query (CARTESIAN PRODUCT in RA)
WHERE	Filters rows (SELECT in RA)
GROUP BY	Partitions data into groups for aggregation

HAVING	Filters groups after aggregation (like WHERE for groups)
ORDER BY	Sorts result by specified attributes (ASC/DESC)
AS	Renames tables or attributes (RENAME in RA)

SQL Operators & Functions

LIKE	Pattern matching (% = any substring, _ = any char)
BETWEEN	Range check: `x BETWEEN a AND b` = `x >= a AND x <= b`
IN	Membership test: `x IN (a, b, c)` or `x IN (subquery)`
EXISTS	Tests if subquery returns any rows (often correlated)
SOME/ANY	Comparison true for at least one value in set
ALL	Comparison true for all values in set
UNIQUE	Tests if subquery has no duplicate tuples
IS NULL	Check if value is NULL (NOT: `= NULL` doesn't work!)
COUNT	Aggregate: count rows/values
SUM/AVG	Aggregate: sum/average of numeric values
MIN/MAX	Aggregate: minimum/maximum value

4.4 SQL Advanced Features (Lec 4)

Database Modification

INSERT	Add new tuples: `INSERT INTO table VALUES (...)`
UPDATE	Modify existing tuples: `UPDATE table SET attr=value WHERE ...`
DELETE	Remove tuples: `DELETE FROM table WHERE condition`
CASE	Conditional logic: `CASE WHEN cond THEN val ELSE val END`

Join Types

NATURAL JOIN
Join on ALL common attributes (can be dangerous!)

JOIN ... USING
Join on specific attributes: `JOIN t USING (col)`

JOIN ... ON Join with condition: `JOIN t ON condition`

LEFT OUTER JOIN
Preserves all tuples from left relation, fills with NULL

RIGHT OUTER JOIN
Preserves all tuples from right relation, fills with NULL

FULL OUTER JOIN
Preserves all tuples from both relations, fills with NULL

Views & Constraints

VIEW Virtual table defined by query: `CREATE VIEW v AS query`

NOT NULL Constraint: attribute cannot be NULL

CHECK Constraint: predicate must be true

FOREIGN KEY
References primary key of another table

CASCADE ON DELETE/UPDATE: propagate changes to referencing tuples

SET NULL ON DELETE/UPDATE: set referencing attributes to NULL

WITH Define temporary relation for single query

4.5 ER Model Terminology (Lec 5-6)

ER Model Basics

Entity Object that exists and is distinguishable (e.g., specific person)

Entity Type Class of entities with same structure (e.g., all instructors)

Entity Set Collection of entities of same type

Relationship Association among entities

Relationship Type
Structure of relationship (drawn as diamond)

Weak Entity Entity that depends on another entity for identification

Attribute Types

Simple Cannot be divided (e.g., age)

Composite Can be divided into parts (e.g., address = street + city + zip)

Single-valued One value per entity (e.g., birth_date)

Multi-valued Multiple values per entity (e.g., phone numbers)

Stored Explicitly stored in database

Derived Computed from other attributes (e.g., age from birth_date)

Discriminator Partial key for weak entity (dashed underline)

Cardinality & Participation

1:1 (One-to-One)
Each entity in A relates to at most one in B, vice versa

1:N (One-to-Many)
Entity in A can relate to many in B, each B to at most one A

M:N (Many-to-Many)
Entities in A can relate to many in B, vice versa

Total Participation
Every entity must participate (double line in ER)

Partial Participation
Some entities may not participate (single line)

Role Function an entity plays in relationship (needed for recursive)

4.6 Physical Design (Lec 8)

B+-Tree Properties

B+-Tree Balanced tree index structure, most widely used in databases

Degree n Maximum number of children per internal node

Internal Node Between $\lceil n/2 \rceil$ and n children

Leaf Node Between $\lceil (n-1)/2 \rceil$ and $n-1$ values

Insert Split Leaf: copy up middle key; Internal: push up middle key

Delete Redistribute (borrow from sibling) or Merge (combine with sibling)

Index Types

Primary Index File ordered by search key (clustering index)

Secondary Index File NOT ordered by search key (must be dense)

Dense Index Entry for every search-key value

Sparse Index Entries for some keys only (ONLY for primary index!)

Hash Index Uses hash function to map keys to buckets

4.7 Query Processing (Lec 9)

External Merge Sort

2-Way Sort Cost: $2N \times (\lceil \log_2 N \rceil + 1)$ I/Os

Multi-Way Sort Cost: $2N \times (\lceil \log_{B-1} \lceil N/B \rceil \rceil + 1)$ I/Os

Phase 1 Create sorted runs of size B pages

Phase 2 Merge (B-1) runs at a time until 1 run remains

Join Algorithms (R outer

Simple NL $|R| + (p_r \times |R|) \times |S|$ - for each row in R, scan all S

Page NL $|R| + |R| \times |S|$ - for each page in R, scan all S

Block NL $|R| + \lceil |R|/(B-2) \rceil \times |S|$ - load many R pages at once

Sort-Merge $3(|R| + |S|)$ (if optimized) - sort both, then merge

Hash Join $3(|R| + |S|)$ - partition both by hash, then match

Query Optimization

Push Selections
Apply WHERE clauses as early as possible (reduce data)

Push Projections
Remove unneeded columns early (reduce tuple size)

Introduce Joins
Replace $\sigma_\theta(R \times S)$ with $R \bowtie_\theta S$

Join Ordering
Execute most selective joins first (smallest intermediate results)

Cost Estimation
Use statistics: $V(A, r)$ = distinct values, n_r = tuple count

4.8 Transactions & Concurrency (Lec 10)

ACID Properties

Atomicity All or nothing - either complete transaction or none of it

Consistency Database goes from one valid state to another (rules never broken)

Isolation Transactions don't interfere with each other (each thinks it's alone)

Durability Committed changes survive system crashes (once saved, stays saved)

Serializability & Conflict

Serial Schedule
Transactions run one after another (no overlap)

Serializable Concurrent but equivalent to some serial schedule

Conflict Two ops conflict if: same data item, different txns, at least one write

Precedence Graph
Draw Ti → Tj for each conflict; no cycles = serializable

Test If graph has cycles → NOT serializable

Locking & 2PL

Shared Lock (S)
For reading - multiple readers OK

Exclusive Lock (X)
For writing - only one writer, blocks everyone

2PL Phase 1 Growing: acquire locks, cannot release any

2PL Phase 2 Shrinking: release locks, cannot acquire any

Guarantee 2PL ensures conflict serializability

4.9 Recovery (Lec 11)

WAL Rule 1 Log before commit - can't say "done" until log is on disk

WAL Rule 2 Log before database write - write to log before writing to DB

Log Record [TID, DID, old, new] - who changed what from which to which

Control Records
[TID start], [TID commit], [TID abort]

Compensation
[TID, DID, old] - record of undo operation

4.10 ER to Relational Mapping (Lec 6)

7-Step Mapping Process M1-M7

M1: Entities Regular entity → table with all simple attributes, PK = entity's PK

M2: Weak Entity
Weak entity → table with FK to strong entity, PK = FK + discriminator

M3: M:N Many-to-many → new junction table, PK = both FKs combined

M4: 1:N One-to-many → add FK to N-side table (NEVER 1-side!)

M5: 1:1 One-to-one → extend total participation side (or merge if both total)

M6: Complex
Recursive: use role names; N-ary: create table with all FKs

M7: Attributes
Multi-valued → separate table; Composite → flatten; Derived → ignore

4.11 Normalization & Functional Dependencies (Lec 7)

Functional Dependencies

FD: → " determines ": same values → same values

Trivial FD (right side contained in left)

Closure + All attributes determined by

Superkey K where K+ = R (determines all attributes)

Candidate Key
Minimal superkey (no subset is also a superkey)

Armstrong's Axioms

Reflexivity →

Augmentation
→ →

Transitivity → and → →

Union → and → →

Decomposition

$\rightarrow \quad \rightarrow$ and $\rightarrow$

Normal Forms: BCNF 3NF 2NF 1NF

1NF	All attributes atomic (no sets, lists, nested values)
2NF	1NF + no partial dependencies (non-key attrs depend on full key)
3NF	For $\rightarrow$: trivial OR is superkey OR in candidate key
BCNF	For $\rightarrow$: trivial OR is superkey (strictest, least redundancy!)

5 Common Confusions & Differences

5.1 Relational Algebra vs SQL

Key Differences

1. **Duplicates:**
   - RA: Relations are SETS → no duplicates
   - SQL: Results can have duplicates (use DISTINCT to remove)
2. **NULL handling:**
   - RA: Typically no NULL values
   - SQL: NULL is a special value; use IS NULL, not = NULL
3. **Operation order:**
   - RA: Inside-out notation: $\Pi_{name}(\sigma_{age>21}(Student))$
   - SQL: Top-down writing: SELECT name FROM Student WHERE age ¿ 21
4. **Natural Join:**
   - RA: Joins on ALL common attributes automatically
   - SQL: Explicit join conditions recommended for clarity

5.2 WHERE vs HAVING

Critical Distinction

	WHERE	**HAVING**
When	Before grouping	After grouping
Filters	Individual rows/tuples	Groups
Can use	Regular attributes	Aggregate functions or GROUP BY attrs
Example	`WHERE salary > 50000`	`HAVING AVG(salary) > 50000`

Rule of thumb: Use WHERE to filter data before grouping, HAVING to filter groups after aggregation.

5.3 SELECT (RA) vs SELECT (SQL)

Confusing Names!

Problem: Same word, different meanings!

Context	Operation	What it does
Relational Algebra	SELECT ()	Filters ROWS
SQL	SELECT clause	Specifies COLUMNS

Remember:

- RA SELECT () = SQL WHERE (filters rows)
- RA PROJECT () = SQL SELECT (chooses columns)

5.4 NULL Value Behavior

NULL is Tricky!

Common mistakes:

1. WRONG: `WHERE attribute = NULL`
   CORRECT: `WHERE attribute IS NULL`
2. **Comparisons with NULL:**
   - `NULL = NULL` → UNKNOWN (not TRUE!)
   - `5 > NULL` → UNKNOWN
   - In WHERE: UNKNOWN treated as FALSE
3. **NULL in aggregates:**
   - Most aggregates (SUM, AVG, MIN, MAX) ignore NULL
   - COUNT(*) counts all rows including NULL
   - COUNT(attr) counts only non-NULL values

5.5 Three-Valued Logic (TRUE, FALSE, UNKNOWN)

Boolean Operations with NULL

A	B	A AND B	A	B	A OR B
TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
TRUE	FALSE	FALSE	TRUE	FALSE	TRUE
TRUE	UNKNOWN	UNKNOWN	TRUE	UNKNOWN	TRUE
FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
FALSE	UNKNOWN	FALSE	FALSE	UNKNOWN	UNKNOWN
UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN

Key Rules:

- TRUE AND UNKNOWN = UNKNOWN
- FALSE AND UNKNOWN = FALSE
- TRUE OR UNKNOWN = TRUE
- FALSE OR UNKNOWN = UNKNOWN
- NOT UNKNOWN = UNKNOWN

In WHERE clause: Only TRUE rows are included; UNKNOWN treated as FALSE!

5.6 JOIN Types: When to Use Which

Join Comparison

Join Type	When to Use	Danger/Caution
NATURAL JOIN	Quick joins on common attrs	Joins on ALL common attrs (may be too many!)
JOIN ... USING	Explicit join on specific attrs	Safer than NATURAL
JOIN ... ON	Full control, any condition	Most flexible, recommended
LEFT OUTER JOIN	Keep all rows from left table	NULLs for unmatched right
RIGHT OUTER JOIN	Keep all rows from right table	NULLs for unmatched left
FULL OUTER JOIN	Keep all rows from both tables	NULLs for unmatched on both sides

Example Problem: student and course both have dept_name

- NATURAL JOIN will join on BOTH ID and dept_name
- Result: excludes students taking courses outside their department!

- Solution: Use JOIN ... ON to specify exact condition

5.7 Subquery Operators: IN vs EXISTS

When to Use Each

	IN / NOT IN	EXISTS / NOT EXISTS
Style	Uncorrelated subquery	Correlated subquery
Semantics	"Is left value in right set?"	"Is right set non-empty?"
Example	`ID IN (SELECT ...)`	`EXISTS (SELECT ... WHERE outer.ID=inner.ID)`
NULL handling	NULL in set causes UNKNOWN	More predictable with NULL
Performance	Can be better for small sets	Often better for large sets

Equivalent queries:

```
-- Using IN (uncorrelated)
SELECT name FROM instructor
WHERE ID IN (SELECT ID FROM teaches);

-- Using EXISTS (correlated)
SELECT name FROM instructor i
WHERE EXISTS (SELECT * FROM teaches t WHERE t.ID = i.ID);
```

Both find instructors who teach at least one course!

5.8 Set Operations Requirements

Union-Compatibility

UNION, INTERSECT, EXCEPT require **union-compatible** relations:

1. Same number of attributes
2. Corresponding attributes have compatible types

Example:
INVALID:

```
(SELECT name, age FROM Student)
UNION
(SELECT course_id FROM Course);  -- Different # of columns!
```

VALID:

```
(SELECT name FROM Student)
UNION
(SELECT name FROM Instructor);  -- Same structure
```

5.9 ER Mapping Confusions (Lec 6)

Common ER Mapping Mistakes

Mistake	Wrong Approach	Correct Approach
1:N FK placement	Add FK to 1-side → redundancy!	ALWAYS add FK to N-side
Weak entity PK	Only use discriminator	PK = FK (strong entity) + discriminator
M:N handling	Try to use FK in one table	ALWAYS create junction table
Multi-valued attr	Store as comma-separated	Create separate table
Derived attr	Store in database	Ignore (compute when needed)
1:1 participation	Random choice of side	Extend total participation side

Key Rule: For 1:N, if you add FK to 1-side, you get update/delete/insert anomalies!

5.10 Normalization: 3NF vs BCNF

Critical Differences

	3NF	BCNF
Rule	→ : trivial OR superkey OR in candidate key	→ : trivial OR superkey
Strictness	Less strict (allows some redundancy)	Stricter (minimal redundancy)
Dependency Preserving	ALWAYS (guaranteed)	Sometimes NOT
Decomposition	Lossless + dependency preserving	Lossless (may lose dependency)
When to use	Need all FDs checkable locally	Can afford JOINs for checking FDs

Example where 3NF BCNF:
instructor(ID, name, dept_name, building) with dept_name → building

- Violates BCNF (dept_name not a superkey)
- Satisfies 3NF IF building is part of some candidate key (depends on other FDs)

5.11 B+-Tree Common Confusions (Lec 8)

Copy Up vs Push Up

Critical Difference - This trips up many students!

	Leaf Node Split	**Internal Node Split**
Action	**COPY UP** middle key	**PUSH UP** middle key
What happens	Middle key stays in leaf AND goes to parent	Middle key moves to parent (removed from child)
Why?	Leaf must maintain all values for range scans	Internal keys are just guides/separators
Example	[12, 15, 16, 17] splits → [12, 15] — [16, 17], copy 16 to parent	Parent [10, 16, 20, 30] splits → [10, 16] — [30], push 20 to grandparent

Memory aid: Leaves COPY (keep the key), Internals PUSH (move the key up)

Dense vs Sparse Index Restrictions

	Primary Index	**Secondary Index**
File order	Ordered by search key	NOT ordered by search key
Can be sparse?	YES	NO (must be dense!)
Why?	Sequential order allows skipping entries	No order, need entry for every value

Common mistake: Trying to use sparse index for secondary index - this NEVER works!

5.12 Join Algorithm Selection (Lec 9)

Which Join Algorithm When?

Algorithm	**Best When**	**Worst When**
Simple NL	Almost never!	Large tables (extremely slow)
Block NL	Small inner, lots of buffer space	Inner table huge, little memory
Sort-Merge	One/both already sorted, need sorted output	Unsorted + no need for sorted output
Hash Join	Equi-join, unsorted data, uniform distribution	Non-equi joins (¿, ¡), skewed data

Golden Rule: ALWAYS use smaller relation as outer in nested-loop joins!

Why Hash Join often wins: $3(|R| + |S|)$ is linear in table size, while Block NL can be quadratic

External Sort: 2-Way vs Multi-Way

Common mistake: Confusing which uses B-1 vs which uses 2

- **2-Way:** Always merges TWO runs (uses 3 buffer pages: 2 input + 1 output)
- **Multi-Way:** Merges (B-1) runs (uses B buffer pages: B-1 input + 1 output)
- **Phase 1:** 2-Way creates runs of size 1; Multi-Way creates runs of size B
- **Passes:** 2-Way needs $\lceil \log_2 N \rceil$; Multi-Way needs $\lceil \log_{B-1} \lceil N/B \rceil \rceil$

Example calculation error: For N=1000, B=100:

- **WRONG:** $\lceil \log_{100} 1000 \rceil$ (using B instead of B-1)
- **RIGHT:** $\lceil \log_{99} \lceil 1000/100 \rceil \rceil = \lceil \log_{99} 10 \rceil = 1$

5.13 Transactions: Conflict Serializability (Lec 10)

Precedence Graph: Cycle Detection

Common Mistake: Wrong edge direction!

- **Rule:** If Ti's operation on X happens BEFORE Tj's operation on X (and they conflict), draw Ti → Tj
- **NOT** Tj → Ti (backward!)

Example:

```
Schedule:
T1: write(A)     ← happens first
T2: read(A)      ← happens second

Edge: T1 → T2 (NOT T2 → T1!)
```

Memory aid: Time flows left to right, arrows point in time direction

Concurrency Problems

Problem	What Happens	Solution
Lost Update	T1's write overwrites T2's write	Use exclusive locks
Dirty Read	T2 reads T1's uncommitted data; T1 aborts	Never read uncommitted (2PL)
Non-Repeatable Read	T1 reads X twice, gets different values	Hold read locks until commit
Phantom Read	T1's query sees new rows second time	Range locking or higher isolation

Key Difference: Non-repeatable read = existing rows change; Phantom read = new rows appear

2PL Confusions

Common Mistake: Early unlock breaks 2PL!

- **WRONG:** lock(A), use A, unlock(A), lock(B), use B, unlock(B)
- **Why wrong?** Unlocked A before acquiring all locks (violated growing phase)
- **RIGHT:** lock(A), use A, lock(B), use B, unlock(A), unlock(B)
- **Rule:** Once you release ONE lock, you can NEVER acquire another

Lock Compatibility Matrix:

T1 holds \ T2 wants	S	X
S	OK	Wait
X	Wait	Wait

Memory aid: Writers are selfish (X blocks everything), readers share (S + S = OK)

5.14 Recovery: Redo vs Undo (Lec 11)

Common Recovery Mistakes

Mistake 1: Only redoing committed transactions

- **WRONG:** In redo phase, skip updates from non-committed transactions
- **RIGHT:** Redo ALL updates (committed or not)
- **Why?** Need to restore database to EXACT state at crash time

- **Then what?** Undo phase removes non-committed changes

Mistake 2: Forgetting compensation records

- **WRONG:** During undo, just set DID = old (no log write)
- **RIGHT:** Set DID = old AND write [TID, DID, old] to log
- **Why?** If crash during recovery, need to know what was undone

Mistake 3: Wrong scan direction

- **Redo:** FORWARD scan (replay history in order)
- **Undo:** BACKWARD scan (reverse changes, last-to-first)

WAL Rule Violations

Why WAL matters:

Violation	What Goes Wrong	WAL Rule
Write DB before log	Can't undo uncommitted changes (no log entry!)	Rule 2: Log before DB write
Commit before log	Can't redo committed changes (log not on disk!)	Rule 1: Log before commit

Golden Rule: LOG FIRST, DATABASE SECOND!

5.15 Functional Dependencies: Common Pitfalls

Closure Computation Mistakes

Mistake 1: Forgetting transitivity
Given: A → B, B → C, compute (A)+

- Wrong: (A)+ = {A, B}
- Right: (A)+ = {A, B, C} (use transitivity!)

Mistake 2: Partial match of left side
Given: AB → C, compute (A)+

- Wrong: Add C to (A)+ (you only have A, not AB!)

- Right: (A)+ = {A} (need BOTH A and B to derive C)

Mistake 3: Thinking superkey = candidate key

- Superkey: ANY set that determines all attrs (can have extras)
- Candidate key: MINIMAL superkey (no proper subset is superkey)
- Example: {ID, name} is superkey but NOT candidate key if {ID} alone is superkey

6 Query Execution: The Process

SQL Query Processing Order

How databases process queries (logical order):

1. **FROM**: Compute Cartesian product of tables
2. **WHERE**: Filter tuples based on condition
3. **GROUP BY**: Partition tuples into groups
4. **HAVING**: Filter groups based on aggregate condition
5. **SELECT**: Project specified attributes
6. **DISTINCT**: Remove duplicates (if specified)
7. **ORDER BY**: Sort result (if specified)

Key Point: This is the LOGICAL order, not necessarily physical execution order!

Writing Queries: Think in Steps

Step-by-step approach:

1. **Identify tables:** What data do you need? Which tables?
2. **Plan joins:** How are tables related? Join conditions?
3. **Filter rows:** What conditions narrow down data? (WHERE)
4. **Grouping:** Need aggregates? Group by what? (GROUP BY)
5. **Filter groups:** Need to filter aggregated results? (HAVING)
6. **Select columns:** What output columns? (SELECT)
7. **Order:** Should results be sorted? (ORDER BY)

7 Common Use Cases & Patterns

7.1 Pattern 1: Simple Filter & Project

Find specific data

Goal: Get names of students older than 21
RA:

$$\Pi_{name}(\sigma_{age>21}(Student))$$

SQL:

```
SELECT name
FROM Student
WHERE age > 21;
```

Pattern: Filter first (WHERE), then select columns (SELECT)

7.2 Pattern 2: Joining Tables

Combine related data

Goal: Find students with their enrolled courses
RA:

$$\Pi_{name,course_id}(Student \bowtie Enrollment)$$

SQL:

```
SELECT s.name, e.course_id
FROM Student AS s, Enrollment AS e
WHERE s.ID = e.student_id;
```

Pattern: List tables in FROM, join condition in WHERE, select needed columns

7.3 Pattern 3: Aggregation

Count/Sum/Average per group

Goal: Count students per department
RA:

$$\gamma_{dept;count(*)}(Student)$$

SQL:

```
SELECT dept, COUNT(*) AS num_students
```

```
FROM Student
GROUP BY dept;
```

Pattern: GROUP BY the category, apply aggregate function

7.4 Pattern 4: Filtered Aggregation

Aggregation with conditions

Goal: Departments with average salary ¿ 50000
SQL:

```
SELECT dept_name, AVG(salary) AS avg_sal
FROM Instructor
GROUP BY dept_name
HAVING AVG(salary) > 50000;
```

Pattern: GROUP BY, compute aggregate, filter with HAVING

7.5 Pattern 5: Set Operations

Combine/Compare result sets

Goal: Courses in Fall 2017 OR Spring 2018
SQL:

```
(SELECT course_id FROM Section
 WHERE semester='Fall' AND year=2017)
UNION
(SELECT course_id FROM Section
 WHERE semester='Spring' AND year=2018);
```

Pattern: Two queries with UNION/INTERSECT/EXCEPT

7.6 Pattern 6: Subqueries

Nested queries

Goal: Students who have taken any course
SQL:

```
SELECT name
FROM Student
```

```
WHERE ID IN (SELECT student_id FROM Enrollment);
```

Pattern: Outer query filters based on subquery result

8 Quick Reference Tables

8.1 Relational Algebra Operations

Operation	Symbol	Description	Commutative?
SELECT		Filter rows by condition	
PROJECT		Select columns, remove duplicates	
UNION		All rows from R or S	
INTERSECTION		Rows in both R and S	
DIFFERENCE	-	Rows in R but not S	
CARTESIAN PRODUCT	$\times$	All (r,s) pairs	*
THETA JOIN		Product filtered by	*
NATURAL JOIN		Join on common attributes	*
RENAME		Change names	-
AGGREGATION		Group and aggregate	-
* Column order may differ			

8.2 SQL Clauses & Operators

Clause/Operator	Purpose
SELECT	Specify output columns
FROM	List tables to query
WHERE	Filter rows before grouping
GROUP BY	Partition data for aggregation
HAVING	Filter groups after aggregation
ORDER BY	Sort results (ASC/DESC)
DISTINCT	Remove duplicate rows
AS	Rename table or attribute
LIKE	Pattern matching (%, _)
BETWEEN	Range: `x BETWEEN a AND b`
IN	Membership: `x IN (a,b,c)` or subquery
EXISTS	Check if subquery returns rows
IS NULL	Check for NULL (not = NULL!)
UNION	Combine results, remove duplicates
INTERSECT	Common rows, remove duplicates
EXCEPT	Rows in first but not second

8.3 Aggregate Functions

Function	Description
COUNT(*)	Count all rows (including NULL)
COUNT(attr)	Count non-NULL values of attr
SUM(attr)	Sum of values (ignores NULL)
AVG(attr)	Average (ignores NULL)
MIN(attr)	Minimum value
MAX(attr)	Maximum value
COUNT(DISTINCT attr)	Count unique non-NULL values

9 Tips for Success

Problem Solving Strategy

For RA queries:

1. Identify needed operations (filter rows? select columns? join?)
2. Start with innermost operation
3. Build outward
4. Check each step has needed attributes

For SQL queries:

1. Identify tables needed
2. Determine join conditions (n tables → n-1 joins typically)
3. Add WHERE conditions for filtering
4. Add GROUP BY if aggregating
5. Add HAVING if filtering groups
6. Specify SELECT columns
7. Add ORDER BY if sorting needed

Common Mistakes to Avoid

1. Projecting before filtering (losing needed attributes)
2. Using = NULL instead of IS NULL
3. Forgetting join conditions (Cartesian product explosion!)
4. Selecting non-grouped attributes with GROUP BY
5. Using WHERE for aggregate conditions (use HAVING)
6. Forgetting DISTINCT when duplicates are unwanted
7. Natural join on wrong attributes (over-constraining)

Optimization Principles

1. **Push SELECT down:** Filter early to reduce data size
2. **Push PROJECT down:** Remove unneeded columns early
3. **Combine SELECTs:** $\sigma_p(\sigma_q(R)) = \sigma_{p \wedge q}(R)$
4. **Join small tables first:** Minimize intermediate results
5. **Use indexes:** B+ trees for ranges, hash for equality

10 Understanding Check: Key Questions

Test Your Understanding

Lectures 2-4 (RA & SQL):

1. What's the difference between (RA) and SELECT (SQL)?
2. When do you use WHERE vs HAVING?
3. What happens when you compare anything with NULL?
4. What's the difference between COUNT(*) and COUNT(attr)?
5. Why is Natural Join dangerous?

Lectures 5-6 (ER & Mapping):

6. What's the difference between an entity and an entity type?
7. Why do you ALWAYS add FK to N-side in 1:N relationships?
8. What's the PK of a weak entity table?
9. How do you map M:N relationships to tables?
10. What do you do with multi-valued attributes?
11. When should you extend which side in 1:1 relationships?

Lecture 7 (Normalization):

12. What does $\rightarrow$ mean (in plain English)?
13. What's the difference between a superkey and a candidate key?
14. What are Armstrong's 3 basic axioms?
15. How do you compute attribute closure (+)?
16. What's the difference between 3NF and BCNF?
17. Why might you choose 3NF over BCNF?
18. What's a lossless decomposition?

Lecture 8 (Physical Design):

19. For a B+-tree with degree n=5, what are min/max children for internal nodes?
20. What's the difference between copy up (leaf split) and push up (internal split)?
21. Can a secondary index be sparse? Why or why not?
22. What's the difference between primary and secondary index?
23. When splitting a leaf node with n values, which key goes to the parent?

Lecture 9 (Query Processing):

24. What's the I/O cost formula for Multi-Way External Merge Sort?
25. Why should you use the smaller relation as outer in nested-loop join?
26. What's the cost of Hash Join?
27. When is Sort-Merge Join preferred over Hash Join?
28. What does "push selections down" mean in query optimization?
29. Which join algorithm works for non-equi joins (e.g., R.a ¡ S.b)?

Lecture 10 (Transactions & Concurrency):

30. What are the four ACID properties? Explain each in one sentence.
31. When do two operations conflict?
32. How do you test if a schedule is conflict serializable using precedence graph?
33. What are the two phases of Two-Phase Locking (2PL)?
34. What's the difference between Shared (S) and Exclusive (X) locks?
35. Can two transactions hold S locks on the same data item simultaneously?
36. What's the difference between dirty read and non-repeatable read?

Lecture 11 (Recovery):

37. What are the two WAL (Write-Ahead Logging) rules?
38. What does a log record [T1, A, 100, 200] mean?
39. In recovery Phase 1 (redo), do you scan forward or backward?

40. In recovery Phase 2 (undo), do you scan forward or backward?
41. How do you decide which transactions to redo vs undo after a crash?
42. What's a compensation log record and when is it created?
43. Why do we redo ALL updates (not just committed) in Phase 1?

If you can answer all these confidently, you understand the complete database systems course!

11 Course Summary: The Complete Picture

Course Coverage - COMPLETE!

All 10 Lectures Integrated:

- Lecture 2: Relational Algebra (, , , , , - operations)
- Lecture 3: SQL Part 1 (SELECT, FROM, WHERE, JOIN, GROUP BY, HAVING)
- Lecture 4: SQL Part 2 (NULL, INSERT/UPDATE/DELETE, outer joins, subqueries, views, constraints)
- Lecture 5: ER Model Part 1 (entities, attributes, relationships, cardinality)
- Lecture 6: ER Model Part 2 (7-step ER→Relational mapping: M1-M7, weak entities, ISA hierarchies)
- Lecture 7: Normalization (FDs, Armstrong's axioms, closure, 1NF/2NF/3NF/BCNF, decomposition)
- Lecture 8: Physical Design (B+-trees, indexing types, hash indexes, storage hierarchy)
- Lecture 9: Query Processing (external sort, join algorithms, cost formulas, query optimization)
- Lecture 10: Transactions & Concurrency (ACID, serializability, conflict, 2PL, locks)
- Lecture 11: Recovery (WAL, redo/undo, log records, 2-phase recovery)

The Complete Database Lifecycle - How Everything Connects:

1. **Design Phase:** Requirements → ER Diagram → Mapping (M1-M7) → Normalization (BCNF/3NF) → Relational Schema
2. **Implementation Phase:** SQL CREATE TABLE → Define constraints (FK, PK, CHECK) → Insert data
3. **Performance Phase:** Create indexes (B+-tree/hash) → Query optimizer chooses join algorithms → Execute with minimal I/O cost
4. **Concurrency Phase:** Multiple users → Transactions (ACID) → Locks (2PL) → Serializable schedules → No interference
5. **Reliability Phase:** Failures happen → WAL logging → 2-phase recovery (redo + undo) → Database restored

Critical Cross-Cutting Insights:

- **Design Performance:** Normalization reduces redundancy BUT increases joins → query optimization becomes critical
- **Logical Physical:** Relational algebra (theory) maps to SQL (practice) which executes via physical operators (B+-trees, hash joins)
- **Concurrency Recovery:** 2PL ensures isolation (concurrency) while WAL ensures atomicity & durability (recovery)
- **Indexes Locks:** Physical design (indexes) speeds up queries; locking ensures correctness with multiple users
- **Theory → Practice → Reality:** RA → SQL → Query plans → Disk I/O → Transactions → Crashes → Recovery

Status: 10 out of 10 lectures integrated (100% COMPLETE!)

This overview Terminology Concepts Reference provides:

- Quick dictionary of all major concepts
- Common confusions and how to avoid them
- Connections showing how topics build on each other
- Understanding check questions covering entire course
- The red thread: from requirements to recovery